Yuno AI Agent Orchestration Platform

Full Technical Documentation — Architecture · Design · ARB Remediation

Version	4.0 (Draw.io-style diagrams)
Date	May 2026
Status	Final — All 8 ARB Issues Resolved
Authors	AI Platform Engineering

ARB Ref	Title	Priority	Status
ARB-001	JWT + API Key Authentication	P0	■ RESOLVED
ARB-002	PostgreSQL Production Backend	P0	■ RESOLVED
ARB-003	Redis EventBus with Fallback	P1	■ RESOLVED
ARB-004	Secrets via pydantic-settings	P1	■ RESOLVED
ARB-005	Rate Limiting (slowapi)	P1	■ RESOLVED
ARB-006	OpenTelemetry Distributed Tracing	P2	■ RESOLVED
ARB-007	GitHub Actions CI/CD Pipeline	P2	■ RESOLVED
ARB-008	Telegram Token Security	P2	■ RESOLVED

1. System Architecture

Four-tier layered view: Client Layer (React + Telegram + REST), API/Orchestration Layer (FastAPI ASGI + JWT auth + rate limiter + WebSocket dispatcher), AI Runtime Layer (LangGraph StateGraph + Agent pool + ToolNode), Infrastructure Layer (PostgreSQL · Redis · Jaeger · GitHub Actions).

Figure 1 — System Architecture

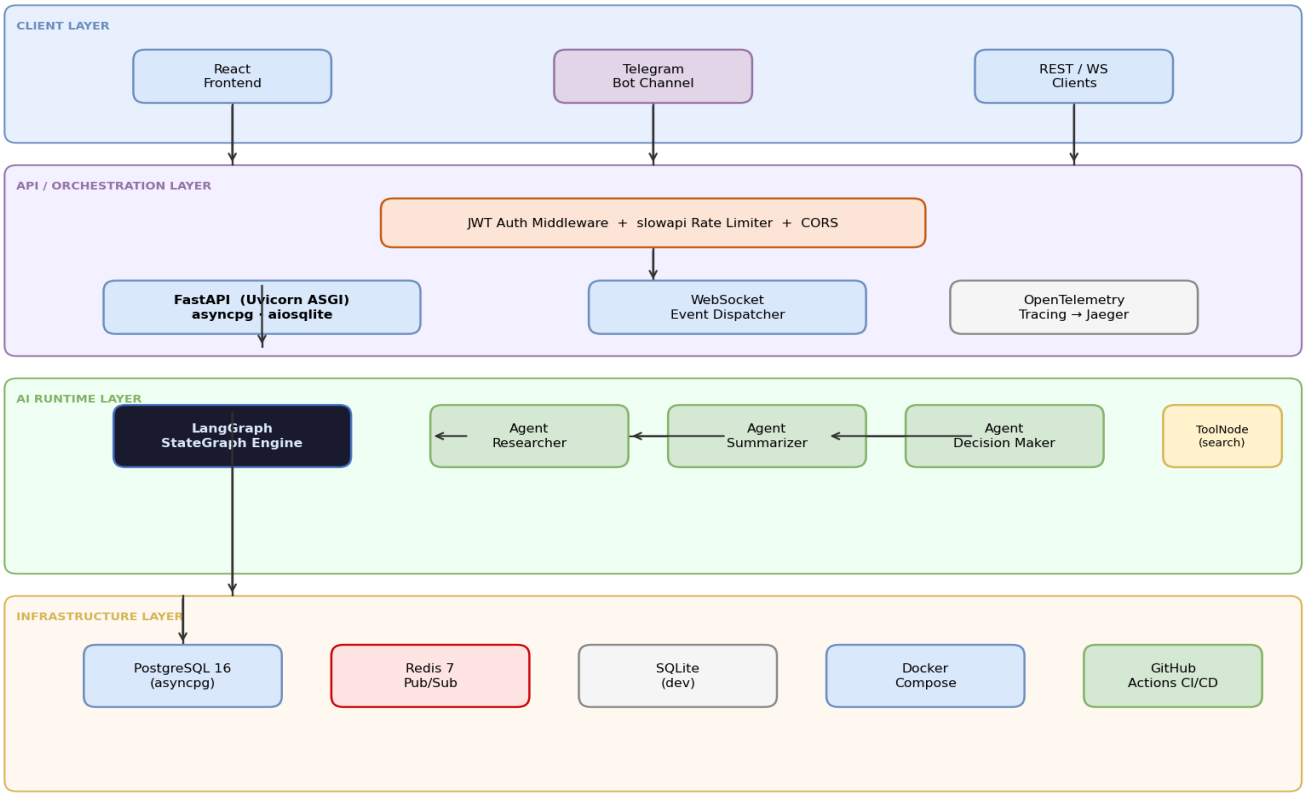


Figure 1 — System Architecture

2. Data Flow Diagram (Level 1)

Five core processes and two data stores. All requests pass through Process 1 (Auth & Route). LangGraph Runtime (P3) communicates externally with OpenAI and Search APIs. Completed run events are published to Redis D2 and consumed by WebSocket subscribers.

Figure 2 — Level-1 Data Flow Diagram

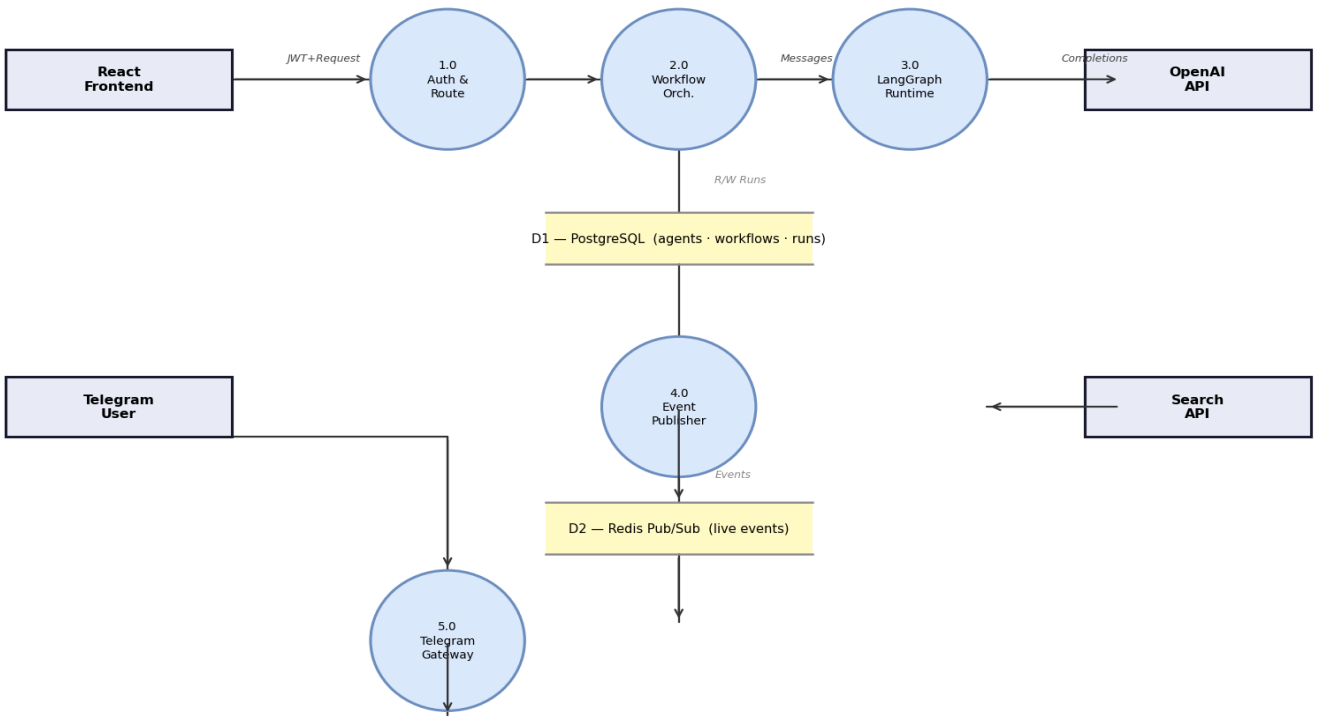


Figure 2 — Level-1 DFD

3. Authentication & Authorisation Flow (ARB-001)

Dual-auth: OAuth2 Bearer JWT (HS256, 480 min TTL) and static API keys. WebSocket auth via ?token= or ?x-api-key= query params; rejected with WS close 4001. Production boot guard in pydantic model_validator blocks default credentials (ARB-004/008).

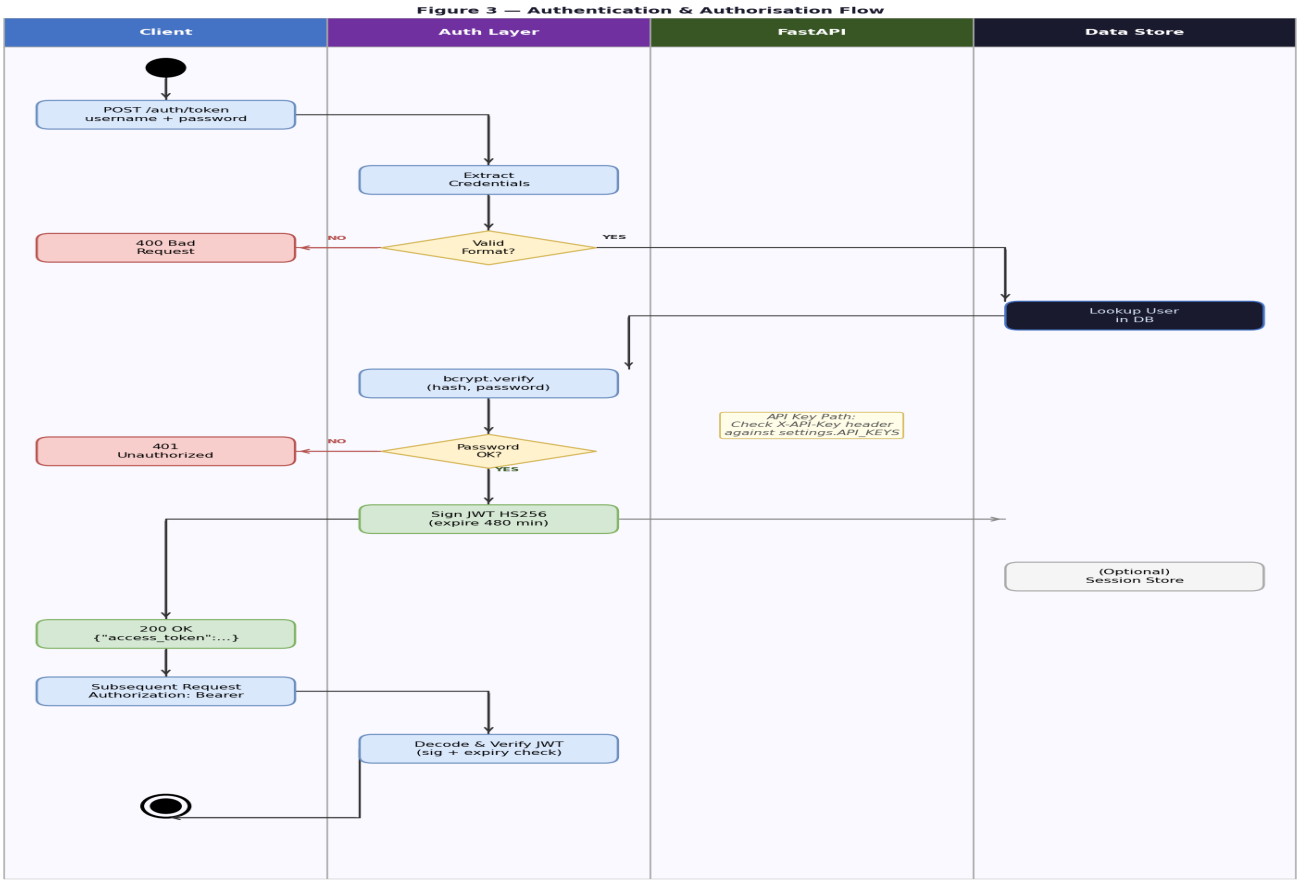


Figure 3 — Authentication Swimlane

4. Activity Diagram — Workflow Execution

Five swimlanes: Client · Auth Layer · FastAPI · LangGraph · Tools. The inner loop (Execute Agent → Tool? → Edge Condition? → Route to Next / END) iterates until the graph reaches END state. Result is published to Redis and returned to the originating client channel.

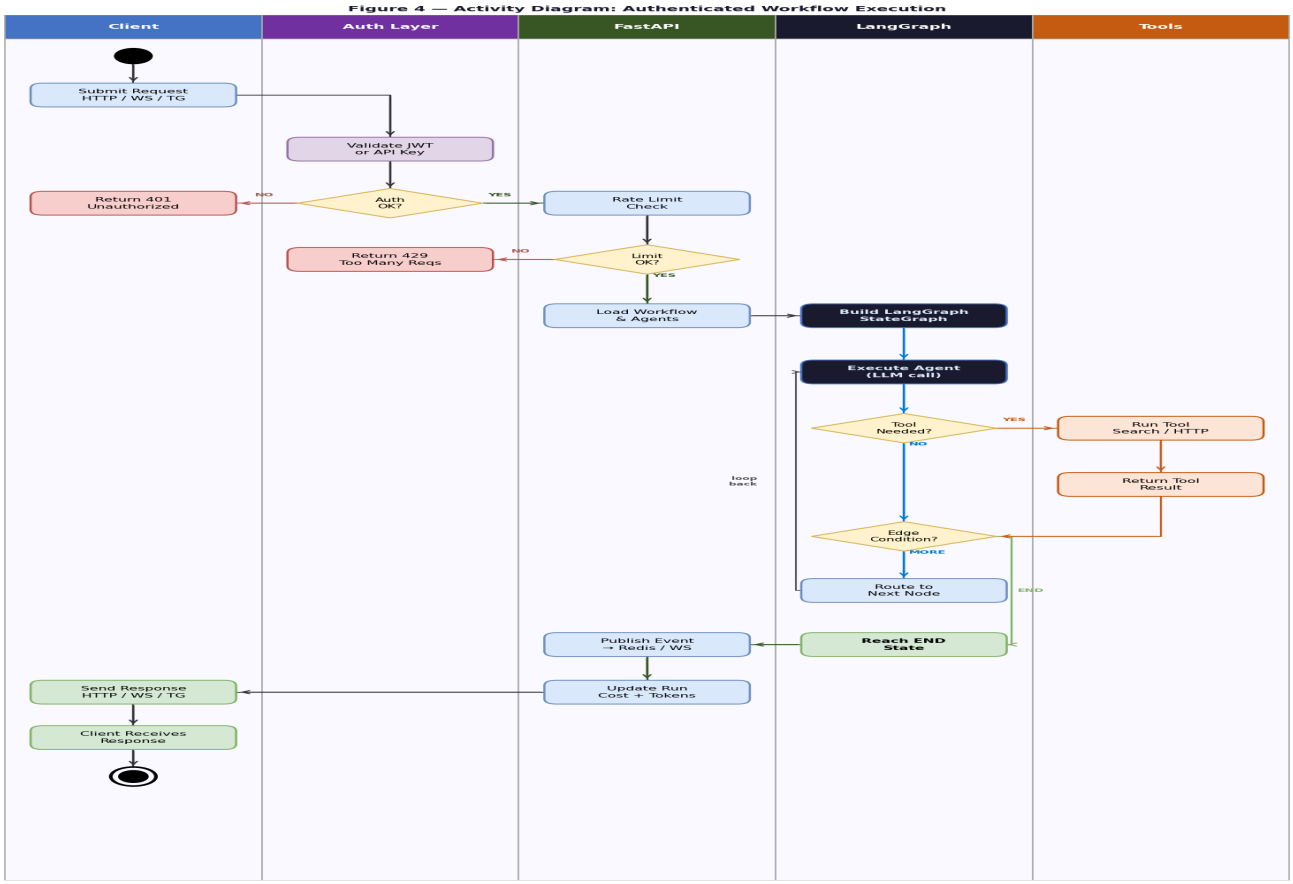


Figure 4 — Activity Diagram (5-lane swimlane)

5. Deployment Architecture

Docker Compose profiles: default (FastAPI + SQLite), production (+PostgreSQL +Redis), tracing (+Jaeger). Nginx handles TLS termination and static SPA serving. GitHub Container Registry stores tagged images; CD deploys via SSH + smoke test.

Figure 5 — Deployment Architecture

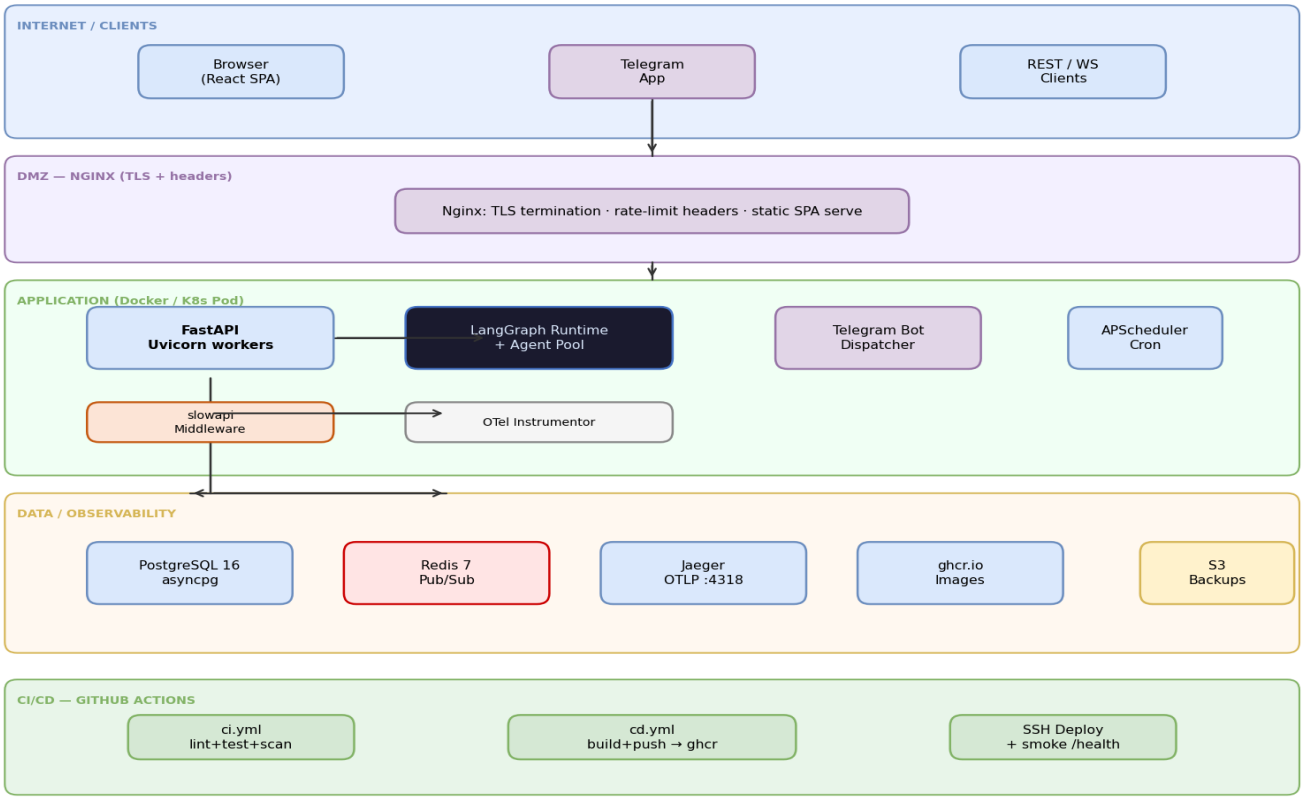


Figure 5 — Deployment Architecture

6. CI/CD Pipeline (ARB-007)

ci.yml: ruff lint → mypy → pytest+Redis svc → pip-audit → ESLint → tsc → build. Gate diamond blocks merge on any failure. cd.yml: Docker build+push → SSH deploy → smoke test /health on every push to main or version tag.

Figure 6 — CI/CD Pipeline (GitHub Actions)

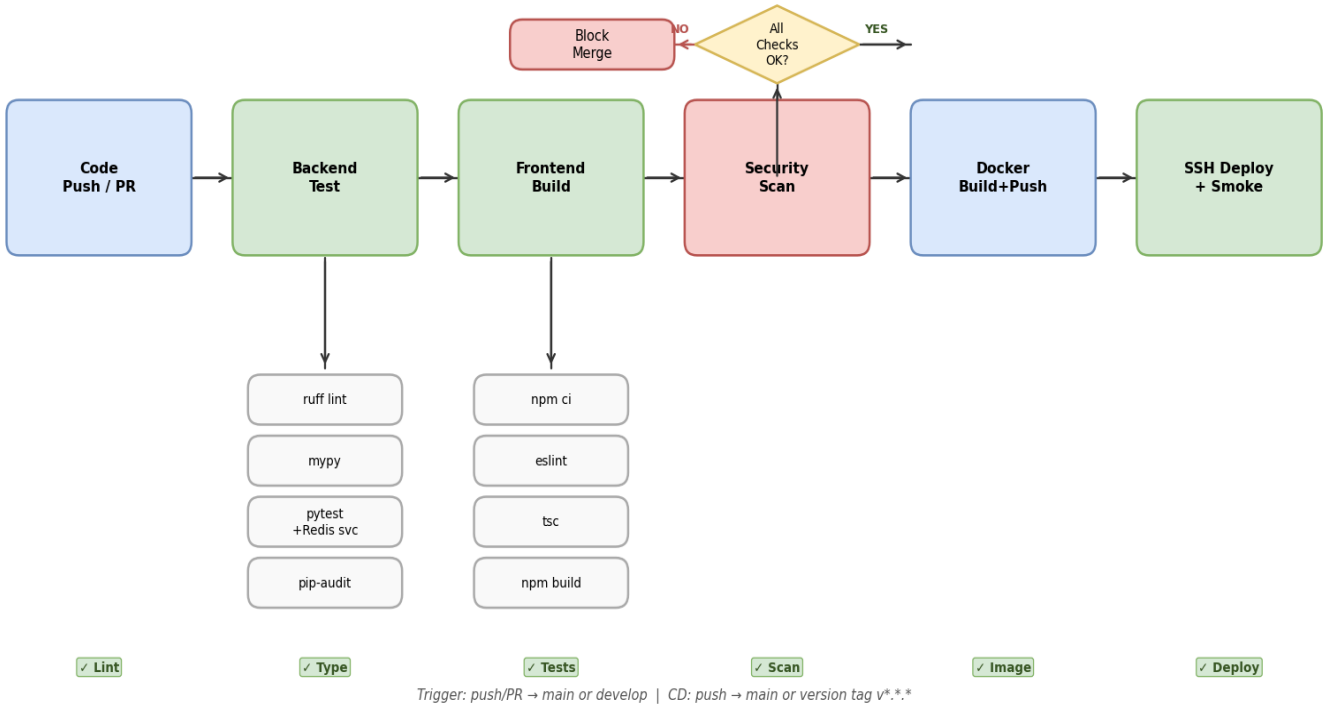


Figure 6 — CI/CD Pipeline

7. Availability Architecture

Multiple Uvicorn replicas behind Nginx health-check. Fully stateless instances share Redis (Sentinel/Cluster) and PostgreSQL (streaming replica). asyncpg connection pool prevents DB exhaustion. Target SLA: 99.9%, RTO < 4 h.

Figure 7 — Availability Architecture

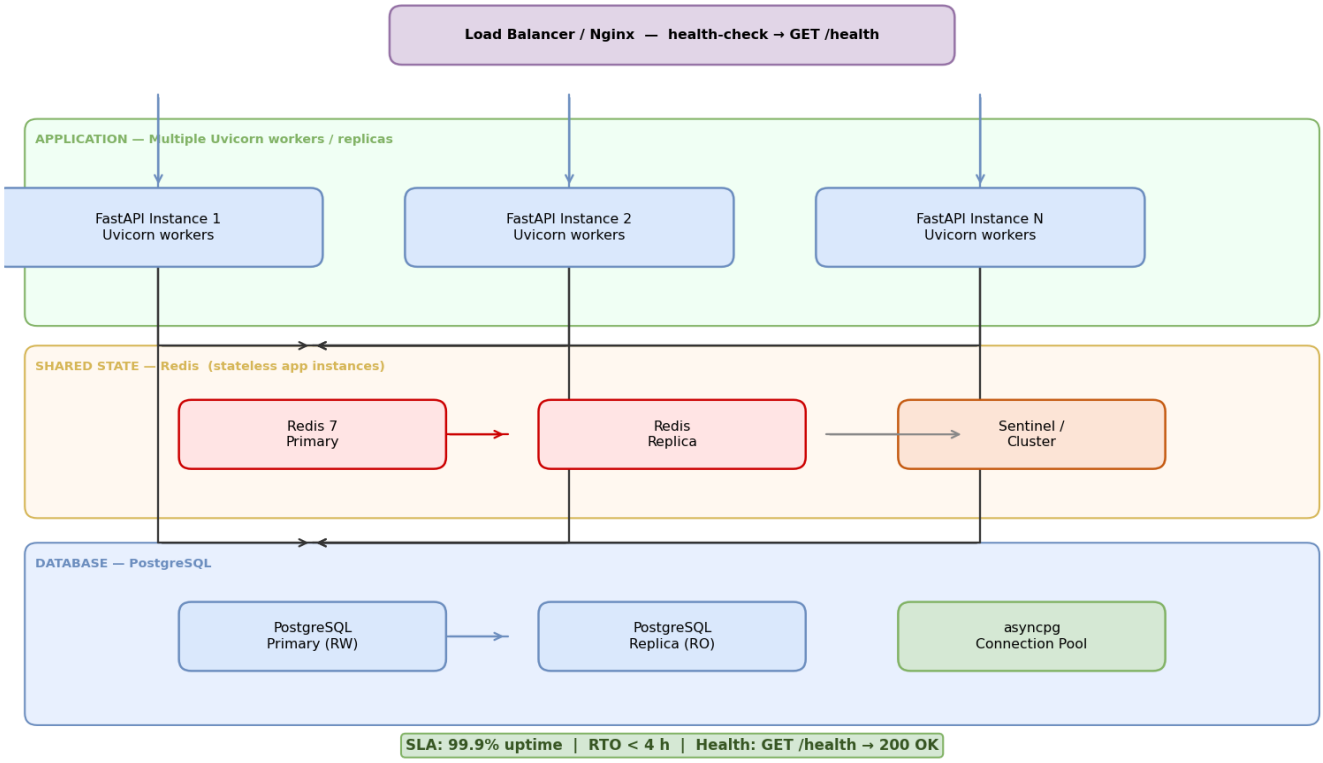


Figure 7 — Availability Architecture

8. Resiliency & Fault Tolerance

Six patterns: circuit breaker for external calls, EventBus fallback (Redis→asyncio.Queue), DB driver fallback (asyncpg→aiosqlite), slowapi rate limiting, LangGraph 25-iteration guard, and pydantic boot guard for secrets safety. All activated via env vars only.

Figure 8 — Resiliency & Fault Tolerance Patterns

Figure 8 — Resiliency Patterns

9. Disaster Recovery & BCP

Six-point timeline: T=0 detect → T+5 triage → T+15 fallback → T+1h RCA → T+4h recovery (RTO) → T+24h post-mortem. Eight recovery steps from Detect to Review. RPO ≤ 1 h · RTO ≤ 4 h · DR drill: quarterly.

Figure 9 — Disaster Recovery & Business Continuity Plan

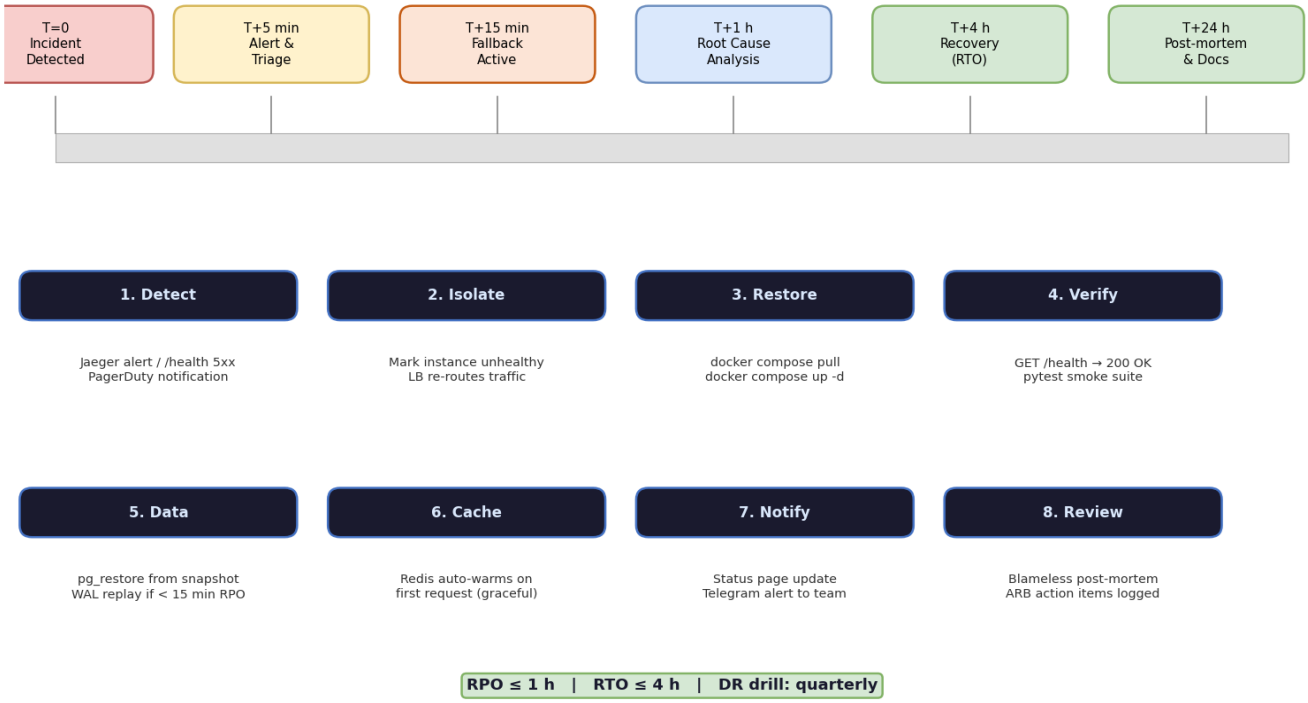
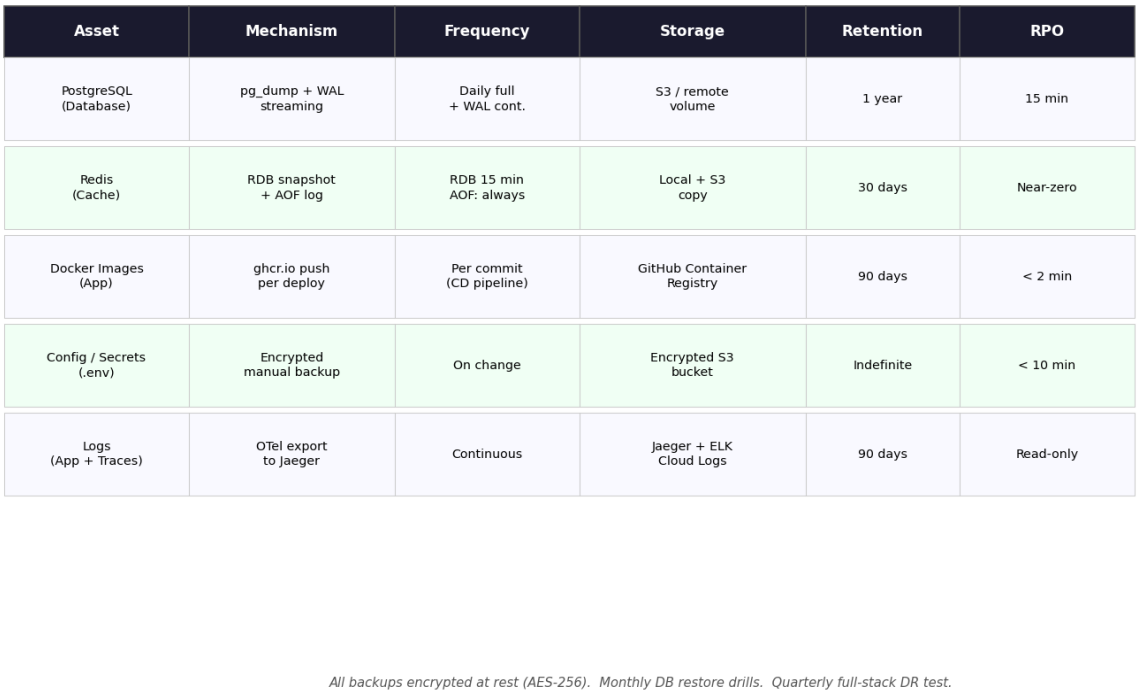


Figure 9 — DR & BCP Timeline

10. Backup Strategy

Five asset classes: PostgreSQL (WAL, RPO 15 min, 1-year retention), Redis (RDB+AOF, near-zero RPO), Docker images (per-commit, 90 days), Config/Secrets (encrypted S3, indefinite), Logs (OTel, 90 days). All encrypted at rest (AES-256). Monthly restore drills.

Figure 10 — Backup Strategy



Asset	Mechanism	Frequency	Storage	Retention	RPO
PostgreSQL (Database)	pg_dump + WAL streaming	Daily full + WAL cont.	S3 / remote volume	1 year	15 min
Redis (Cache)	RDB snapshot + AOF log	RDB 15 min AOF: always	Local + S3 copy	30 days	Near-zero
Docker Images (App)	ghcr.io push per deploy	Per commit (CD pipeline)	GitHub Container Registry	90 days	< 2 min
Config / Secrets (.env)	Encrypted manual backup	On change	Encrypted S3 bucket	Indefinite	< 10 min
Logs (App + Traces)	OTel export to Jaeger	Continuous	Jaeger + ELK Cloud Logs	90 days	Read-only

All backups encrypted at rest (AES-256). Monthly DB restore drills. Quarterly full-stack DR test.

Figure 10 — Backup Strategy